带因果的结构 IR

容器面与状态面：提升 AI 生成 RTL 的能力，实现可追溯的 AI 验证

组会汇报 · 2026-11

背景 ①：模拟器把“指令”和“部件”的关系藏在了代码里

硬件实际工作时：**指令是灵魂，部件是身体**——指令依次流过各个部件，部件的位置和状态决定指令哪一拍能走

模拟器	组织方式
GPGPU-Sim / Accel-Sim	GPU 周期级模拟器； route B 用它的 FML 移植版
gem5 （ MinorCPU / O3CPU ）	最常用的 CPU 模拟器
CVA6 官方性能模型	Python ，约 670 行；我们的实验对象
Vortex simX	开源 GPGPU 的 C++ 周期级模拟器，与官方 RTL 同仓库
PyMTL3	功能级 / 周期级 / RTL 三层，周期级与 RTL 不要求一一对应

以 CVA6 模型为例（我们逐条核对过）

19 条因果边里， **13 条**只由函数调用顺序或端口先后决定，代码里没有一句话表达这个意图

每类指令发射前只**检查**一个部件，却**占用**好几个：单向挡住，读代码时很容易看漏

注释说乘除与第二 ALU 是同一部件，代码行为并非如此

模拟器有部件也有指令，但“**哪条指令在哪一拍、经过哪个部件、挡住了谁**”没有写成可检查的规格

例子： CVA6 模型里“谁挡住了谁”藏在两段代码的组合里

双发射（每拍最多发两条），实验 2 的真实记录；两段程序只差指令顺序；右侧方框是发射的拍

情况 A： 先加载，后分支

#8	lw （加载）	第 4 拍
#9	bne （分支）	第 4 拍

同一拍都发出去了；这一拍“存储单元”上有两个占用者

情况 B： 先分支，后存储

#8	bne （分支）	第 4 拍
#9	sw （存储）	第 5 拍

记录：第 4 拍被“存储单元”挡住，挡住它的是第 8 条（分支）

```
def issue_branch(self):    #  分支发射时
    self.alu = True
    self.branch = True
    # Stores are not allowed yet
    self.stu = True          #  也标“存储单元”忙

def issue_ldu(self):       #  加载发射时
    self.ldu = True
    self.stu = True          #  也标“存储单元”忙
```

发射**前**：每类只查自己那一位——存储查“存储单元”，分支只查“分支单元”

model.py 第 290–300 、 323–335 行

1 挡住是单向的

分支挡得住存储；加载、存储都挡不住分支

2 意图看不出

唯一的说明是一句注释“ Stores are not allowed yet”

IR 里明确写成：分支对存储单元“**只占用、不检查**”，存储对它“**检查并占用**”

背景 ②：把微架构明确写出来，AI 就写得更好

1 HINT （arXiv 2608.07625）的启发

在规格与 RTL 之间加一层可执行的中间层，写明状态归属、提交纪律；AI 先写这一层，再照着写 RTL

2 Representation Bottleneck （arXiv 2604.17097）

6 种硬件语言、 202 道单模块题： HLS C 只有 **3%**， Verilog **88%**；换模型差别不到 1.25 倍。 HLS 把时序和接口交给工具决定，AI 看不到也控制不了每拍的行为

3 FML （合作者）

按部件建模，读别的部件时必须写明读本拍值还是上一拍值——和中间 IR 一样，体现的是实际微架构

我们比 HINT 多了什么

HINT 是**事务级**，原文明确不做周期精确；我们写明本拍还是下一拍

HINT 没有容器面 / 状态面和因果边；出错时只能说“错在哪一层”

我们能追到**哪个部件、哪一面、哪一拍、被谁挡住**

启发： AI 需要看到**周期级的微架构信息**，但最终仍写它最擅长的 Verilog

问题①：结构越复杂，AI 越写不对

Benchmark	考的是什么	结果
VerilogEval v2 （NVIDIA， 2024）	单个模块，按描述写 RTL	最好的模型通过率 **89–98%**，基本饱和
ArchXBench （MLCAD'25）	6 级、 51 题：基本运算 → 简单流水线 → 迭代算法 → 深流水线信号处理 → 图像 / ML → 子系统	前 3 级最好的模型做对 16/30；**第 4 级起基本全部失败**
CVDP （NVIDIA， 2025）	783 题、 13 类：写 RTL、写验证、修 bug、对规格	代码生成 pass@1 最高约 **34%**；最弱的是**写验证和检查**

流水线里，一条指令能不能前进，取决于它和 scoreboard、功能单元、队列、仲裁之间的关系——这正是单模块 benchmark 考不到的

难点已经从“写对一个模块”，转到**处理好指令与结构之间的关系**

问题 ②：只看测试结果，AI 改不到地方

别的领域的证据

1 SWE-bench （Wang 等 ICSE'26）

AI“已解决”的补丁约 **11%** 不正确

2 自动程序修复（Qi 等 ISSTA'15）

通过测试的补丁约 **95%** 是错的，多是删掉功能

3 香山处理器（VSpector）

25 个 bug 中 **9 个**是 RTL 与参考模型错得一样，测试发现不了

4 硬件仓库修 bug （HWE-Bench）

最好 70.7%， SoC 级 **< 65%**；难在**找根因**和懂硬件规矩

我们的例子： route B （基于开源 GPGPU-Sim）

AI 分别生成两个模块，各自逐拍比对全过；接起来，第 5051 拍不一致。
真正原因：接口少了一条“本拍更新后的值”

给 AI 的反馈	会把 AI 引到哪
测试结果：“第 5051 拍不一致”	在报错处打补丁
第一处分歧：“下游模块输入不一致”	去改没错的下游模块
按模块归因（ChatCPU）	改后放进去的模块——其实两个都没错
带因果的 IR：“这条同拍边在接口上没导出”	**改接口**

只靠测试，AI 被引向**报错的位置**；沿着指令与结构之间的因果，才能指向**出错的来源**

核心思想：加入因果，把结构拆成容器面与状态面

容器面：占位置
有限的位置，被占了别人就得等
例： scoreboard 最多 8 条；功能单元一拍只接一条
追问：现在是谁占着？什么时候走？

状态面：存信息
被写、被读，影响后面的决定，但不让人排队
例：寄存器是否准备好；分支历史表
追问：上一次是谁、哪一拍改的？

1 一个结构可以两面都有

scoreboard 既有 8 个位置（容器面），又记着每个寄存器还有谁没写完（状态面）；一次提交同时改变两面

2 指令是在结构里流动的“灵魂”

每类指令按路线依次使用各个结构；用五项规矩写清怎么用：几个位置、占多久、在哪判断、知道什么、多人抢时给谁

硬件 = **指令在使用结构做事**；把这层关系写清楚， AI 才能在复杂处理器上写对

因果边：一个动作经过某一面影响另一个动作

每条边标明“本拍就看到”还是“下一拍才看到”。例：一条乘法后紧跟一条用它结果的加法

	第 t 拍	第 t+1 拍	第 t+2 拍	第 t+3 拍
mul a0, s0, s1	发射	执行中	算完	提交
add a1, a0, s2		等待	发射	算完
功能单元（容器面）		被乘法占着		
a0 是否就绪（状态面）		没有	本拍算完即可用	

1　加法的一次等待，记下两个原因、各自“被谁挡住”

容器面：功能单元被乘法占着；状态面： a0 的写入者是那条乘法，还没算完

2　“本拍算完、本拍可用”在模拟器代码里没写

只是函数调用顺序带出来的——这类隐含的因果，正是 IR 要显式写出来的

模拟器按功能组织代码，因果**藏在调用顺序里**； IR 把它变成**可检查、可追溯**的边

用途一：提出一种方便 AI 生成的模拟器范式



范式包括什么

1 按三类路径分开写

固定功能路径：只写延迟 / 吞吐；非固定路径（访存）：事件模型；指令路径：按结构逐个写

2 指令路径按 IR 组织

每个结构一个模块，分容器面与状态面；状态分寄存器（下一拍）与本拍信号（本拍）

3 因果不许藏在代码顺序里

每处读都对应 IR 的一条边；检查与占用分开写

为什么方便 AI 生成

复杂处理器的难点（指令与结构的关系）已经在模型里写清， AI 只需“翻译”

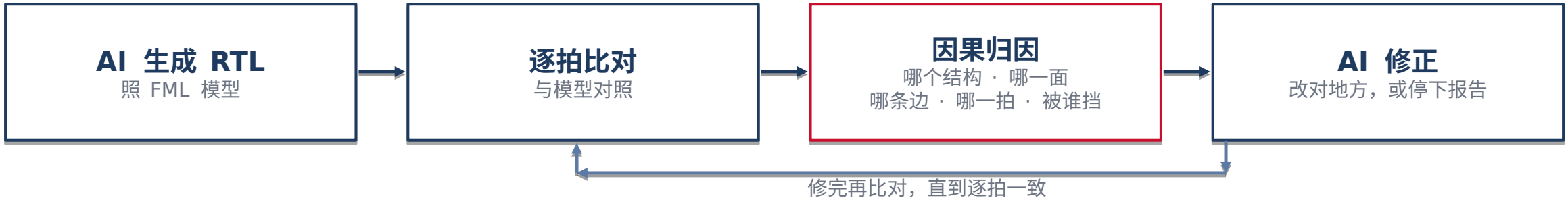
本拍 / 下一拍直接对应组合逻辑 / 触发器

模型可运行，生成的 RTL 能逐拍比对，出错能沿边归因

先例要引用： PyMTL3 、 Cascade （周期模型细化成 RTL ）； Kôika / BSV （直接编译）

目的：让 AI 在**复杂处理器**上也能生成 RTL ，并且**更快、更好**（待生成实验证明）

用途二：可追溯的验证，让 AI 检查并修好自己生成的 RTL



追到的来源	AI 怎么做
① 结构内部实现错 / ② 接口、同拍与寄存弄反	修
③ 打破了别人依赖的时序规矩	修，或登记修改
④ 规格或参考错 / ⑤ IR 漏了依赖 / ⑥ 追不下去	**停下，报告给人**

期望效果

AI 不再只知道“错了”，而知道**错在哪、为什么**，能检查自己生成的 RTL

修正迭代更少，假修复（凑出“通过”）更少，该停时会停

从而**提升 AI 生成 RTL 的效果**

衡量：有 / 无因果归因时，修正成功率、迭代次数、假修复率

两个用途合成一个闭环：**范式让 AI 生成，因果让 AI 自查自纠**——共同提升复杂处理器上的生成效果

和已有工作的关系

工作	有什么	它没做的（我们要补的）
HINT （2608.07625）	规格与 RTL 之间的可执行中间层；状态归属	不做周期精确；报错只到“哪一层”
ChatCPU （DAC'24）	周期精确模型作参考，逐模块生成	每模块一个延迟；参考与 RTL 同源；归因到“新模块”
HLS （Vitis HLS、 Catapult）	输入不带时序的 C 算法，工具决定每拍做什么	时序藏在工具里；我们反过来：**时序由模型规定**
BSV / Kôika / Assassyn	同拍 / 寄存是语言概念，写清后直接编译	须从头用该语言写；不检查 AI 写的 RTL、不归因

Zeller 因果链（FSE'02； Cleve & Zeller ICSE'05）：用“对比两次运行”找出错的原因

做法：拿一次正确、一次出错的运行，把两边的程序状态一半一半地互换、重跑，缩小到“换了它，错误就消失”的那几个变量；再在时间上追，看原因从哪个变量转移到哪个变量

长处：不需要任何规格，只要能重跑；局限：不知道系统该怎么工作，每缩小一步都要重跑一次。我们有 IR 写明谁影响谁，**沿因果边直接往回找**；实验中把它作为对照组

我们的位置：**时序由模型规定，实现由 AI 完成，正确性由因果 IR 检查**

验证这个思想：在 CVA6 上从模拟器得到因果 IR

1　平台： CVA6

开源 RISC-V，官方带 Python 周期级性能模型和可综合 RTL

2　只看模型、逐条数出 IR

14 个结构（10 个有容器面、5 个有状态面）、19 条因果边、10 种等待原因

3　三方向检查

没漏画 · 每次等待能解释 · 只用 IR 的规则能预测模型

1,312

次运行，0 次解释不了的等待

19 / 19

条因果边都被触发过

100%

12.5 万条指令的发射拍，只用 IR 就全部推对

结论：容器面 / 状态面 + 因果边的写法，**足以完整描述**这个模拟器的时序行为

怎样保证因果 IR 不出错：三道检查 + 四条修正规矩

① 代码 → IR：数全了吗?
模型里每个状态都归到某个结构
绑定由程序从模型中抽取、核对
检查器自动查 IR 自身的规则

② 运行 → IR：解释得了吗?
每一拍、每次等待都要用 IR 的原因解释
每条原因在那一拍必须单独成立
每条性质在每一拍都要成立

③ IR → 运行：预测得准吗?
不调用模型，只按 IR 的规则
推出每条指令在哪一拍发射
推错 = IR 缺了或写错了规则

发现不一致后，按四条规矩修正

1 有证据
精确到哪个用例、哪条指令、哪一拍，以及模型代码的哪几行

2 证明必要
把新规则去掉或反着写，必须有用例变错（例：去掉对齐规则，264 次运行不再一致）

3 检查器也要被检查
故意注入错误，检查器必须报出（阳性对照），否则“0 失败”不可信

4 只加不删、全量重跑
旧条目标“已被取代”，保留修订历史；每次修订后重跑全部检查

修正不是失败：说明这套规则**能自己发现 IR 的错**，而且每次修正**有据可查、可复现**

7 次修正：每一次被哪道检查抓到

修正了什么	被谁抓到	怎么证明改对了
预测错误的惩罚可以中途变短（ #1 ）	② 运行解释：说好清空 6 拍，一条分支只等了 1 拍	换回旧说法，检查报出 2 次违反
事件记录的闸门按种类判断（ #2 、 #3 ）	核对模型代码与统计： #1 里有一句话错了	那句话来自**复核意见**，没核对代码
指令队列发射时的对齐（ #4 ）	③ 预测： 307 对两两间隔推错	去掉这条规则， 264 次运行不再一致
“ 检查”与“占用”分开（ #5 、 #6 ）	③ 预测：剩下的 38 对全是这一类	约定升到 v3 ；顺带抓出**我们自己的工具**记了虚假等待原因
四个隐含假设写进 IR （ #7 ）	预测器用了、 IR 没写的假设	每条反着写，例： WAW 一条有 71 次运行不再一致

4,871
修正前，只用 IR 预测对的两两组合

→

5,178
补上对齐规则（ #4 ）

→

5,216 / 5,216
分开检查与占用（ #5 、 #6 ），全部推对

错的不只是 IR： **复核意见**和**检查工具**也被这套流程抓出过错，所以流程本身要可检查

下一步

1 按 IR 设计 FML 模型：把因果关系融进模拟框架

实现框架的同时内建因果记录，看能否做到：仿真速度不明显变慢，又能作为 AI 生成 RTL 的规格

2 对比三种生成方式，找出效果最好的

模拟器 → RTL 、模拟器 + 因果 IR → RTL 、 FML + 因果 IR → RTL ： 比一次做对的比例、迭代次数、假修复

3 走向更复杂的处理器流水线

从 CVA6 的顺序发射，扩展到更复杂的流水线（多请求方仲裁、 credit 等），看效果能否保持

看因果的**实际效果**：帮生成有效 → 走 **IR + FML → RTL** + 验证；不明显 → 重心转向**可追溯验证**

想和大家讨论的问题

1 **容器面 / 状态面的划分合理吗?**

一个结构可以两面都有；分开是为了追因果时往不同方向追。有没有它覆盖不了的硬件结构?

2 **为什么不直接编译?**

时序已写清， BSV / Kôika 可直接编译出 RTL 。我们强调抽象、实现选择与检查归因，够不够?

3 **下一个平台**

CVA6 没有多请求方仲裁和 credit 。下一个用 Vortex 还是其他 GPU 设计?